Inference Engineering × Applied AI

Presented by Danial Hasan × Pillars Network

Danial Hasan

Member of Technical Staff · Appear

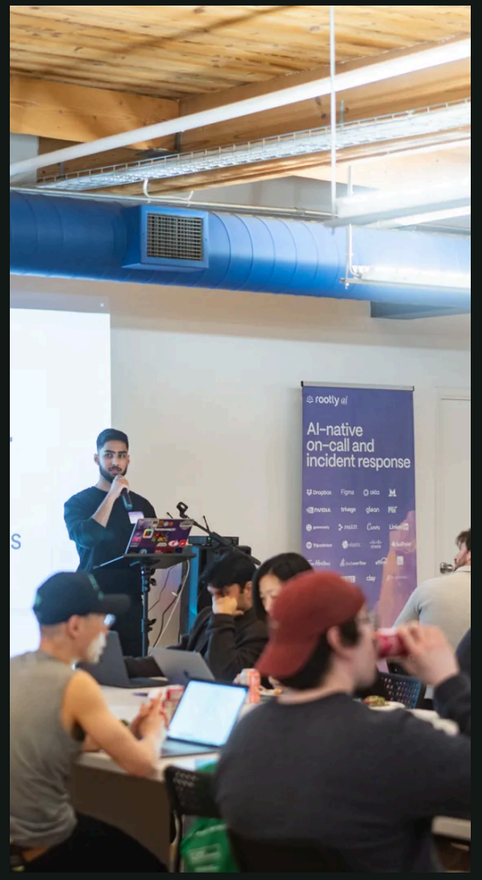
Who I am

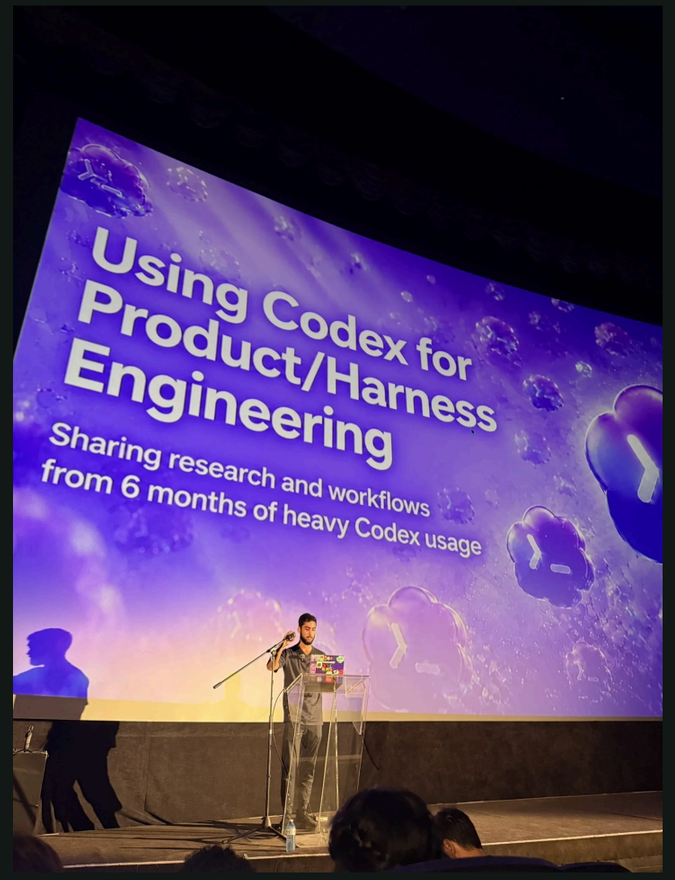
- Started programming at **13**.
- Turned down a Waterloo offer to work full time at **Compass Digital**, building software for restaurant operators to manage logistics and operations.



Who I am

- Moved to downtown Toronto about three years ago.
- Led workshops on Claude Code, agent orchestration, harness engineering, and mechanistic interpretability; collaborated with **AWS in 2025**.
- Built **2 startups**, started an **AI consulting business**.
- Now **Member of Technical Staff at Appear**, a growth engineering startup.





What we'll cover

1. Workloads
2. Service commitments ($$$)
3. Ontologies
4. Hardware
5. Prefill and decode
6. Inference engineering levers
7. Applied AI levers
8. Putting it together

Then, we'll apply these ideas in the workshop.

Get ready before the lecture

github.com/danialhasan/inference-engineering-workshop

Give your coding agent the repo and this instruction:

> “I’m at an inference engineering workshop. Read this repository and follow its setup instructions on this machine. Check that the local model and dashboard work, and fix any setup issues. Leave the exercises ready for me.”

Ready means: Qwen2.5 1.5B runs locally, a live test finishes, and the dashboard is open.

Lecture

Workloads, systems, and engineering decisions.

Workloads

The work people need done determines what the system must deliver.

Inference engineering and applied AI

Inference engineering

Build the systems that run trained AI models.

Make model responses fast, reliable, and efficient.

Example: the server that runs a model.

Applied AI

Use AI models to solve a problem.

Build the application, connect its tools, and check its results.

Example: a coding agent.

Inference engineering and applied AI

Company	Inference engineering	Applied AI
Baseten	Model serving	—
Modal	GPU compute and model serving	—
Groq	Inference hardware and API	—
Together AI	Shared and dedicated model serving	—
Fireworks	Model serving and optimization	—
Cursor	Composer serving	Editor and coding agent
OpenCode	Zen · managed model access	OpenCode · coding agent
Factory	Managed inference	Droid · coding agent
CodeRabbit	—	Code review
Harvey	—	Legal research and document work

Selected workloads. One company can work in both areas.

It's all about the workload

Applied AI

Define the task.
Check the result.

→

The workload

What inputs?
How much output?
How many requests?

→

Inference engineering

Run that work within its time, cost, and quality requirements.

“Every inference engineering decision you make will be downstream of your use case.”

Philip Kiely, *Inference Engineering* (2026), §1.2 “About Your App,” p. 27

Workloads in the industry

Company	Workload	Input → output	What matters?
Baseten	**Model serving** An app asks a model to summarize a document.	Model requests → generated responses	Response time, capacity, and cost.
Modal	**Batch inference** Classify 10,000 support tickets overnight.	Many input records → model results	Finish by the deadline at a low cost.
Cursor	**Coding agent** Fix a login bug and run the tests.	Task + code + tool results → code changes	Complete the task correctly across many calls.
CodeRabbit	**Code review** Check a proposed code change for bugs.	Code changes + context → review findings	Find real problems with few false alarms.
Harvey	**Legal research** Find when a customer can cancel a contract.	Question + sources → answer with citations	Use the right sources and support the answer.

Workloads in this room

Audience project	Possible model workload	What matters?
RAG pipeline on AWS	Question + retrieved text → answer	Relevant sources and response time
Enterprise AI research	Documents → report with sources	Long inputs and supported claims
UN contract scraper	Contract pages → structured fields	Accurate extraction at volume
AI damage assessment	Images → damage assessment	Image detail and accuracy
IDS anomaly detection	Network activity → anomaly score	Rare threats and false alarms
Calendar, Notion, and home agents	Request + tool results → next action	Correct actions across repeated calls

MCP servers and the 900 GB streaming platform provide infrastructure.
Any model calls they make create an inference workload.

Service commitments

Turn customer needs into measurable promises, costs, and consequences.

An example enterprise service level agreement

The work covered

One request = one report job:
one question + source documents → one report with citations.

Each job: up to **16,000 input tokens** and **1,000 output tokens**.

The application handles up to **20 report jobs at once**, including queued jobs.

Example fee: **US$20,000 per month**.

The monthly commitments

95% of eligible requests return a complete report within **30 seconds**.

99.9% of eligible requests succeed within the **60-second timeout**.

If either target is missed, a **5% service credit** means **US$1,000** toward a future bill.

The measurement is the service level indicator (SLI), and its target is the service level objective (SLO).
The agreement and its consequences form the service level agreement (SLA).

Terminology: Google, Site Reliability Engineering, chapter 4. Terms and fee above are illustrative.

Real prices and service credits

OpenAI · GPT-6.1 Sol

Standard pricing per million tokens:
US$2 input · US$10 output.

Our 16,000-input + 1,000-output job:
US$0.042 per report.

100,000 reports per month:
US$4,200 in model usage.

Baseten · published credit tiers

For an illustrative **US$100,000 monthly bill** for the affected service:

Monthly availability	Credit
99.5% to below 99.9%	**US$10,000** · 10%
99.0% to below 99.5%	**US$15,000** · 15%
Below 99.0%	**US$30,000** · 30%

These are different providers' terms. Neither promises our complete application will respond in 30 seconds.

Sources: OpenAI Standard pricing · Baseten service level agreement, checked November 10, 2026.
US dollars. Model-cost example excludes caching, tools, retries, taxes, and other application costs. Baseten bill size is assumed.

What does that promise require?

Requirement	Engineering decision	Evidence we need
A complete report in 30 seconds	Budget time for retrieval, queues, and generation.	End-to-end latency, including the slowest 5%.
Long inputs and 20 requests in flight	Size memory and capacity; tune batching against queue delay.	A load test with these input and output sizes.
99.9% successful responses	Plan spare capacity, failover, and bounded retries.	Failure tests and monthly service measurements.
Supported answers at a viable cost	Evaluate model choice, retrieval, and caching together.	Answer quality and cost per successful report.

A faster model call helps only if the complete service meets its requirements.

Kiely, *Inference Engineering* (2026), §1.4.2, p. 37; §4.5, pp. 112–114.
Engineering choices are proposed responses to the fictional SLA, not measured guarantees.

Where do the 30 seconds go?

An example budget: 24 seconds of work, with 6 seconds of headroom.



Stage	Budget	What happens
Retrieve sources	3 seconds	Find the documents needed for the answer.
Wait for model capacity	2 seconds	The request waits in a queue.
Process the input	4 seconds	The model reads the prompt and sources: prefill.
Generate the report	14 seconds	The model produces the output tokens: decode.
Validate and send	1 second	Check the response format and return the report.
Headroom	6 seconds	Allow for variation across requests.

Illustrative allocation, not a measurement. Time runs from service receipt to complete response.

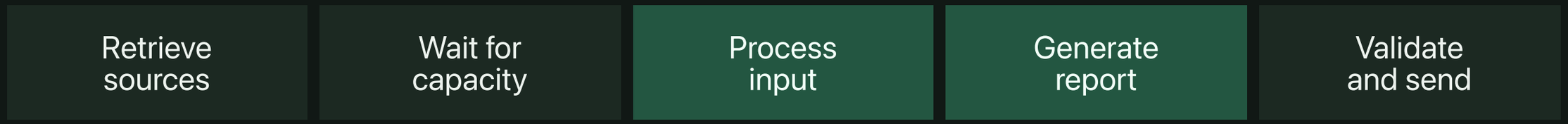
Kiely, *Inference Engineering* (2026), §1.4.2, p. 37.

Ontologies

The parts of inference systems, AI applications, and the businesses around them.

Two layers, different levers

Applied AI owns the complete application workflow



Applied AI levers

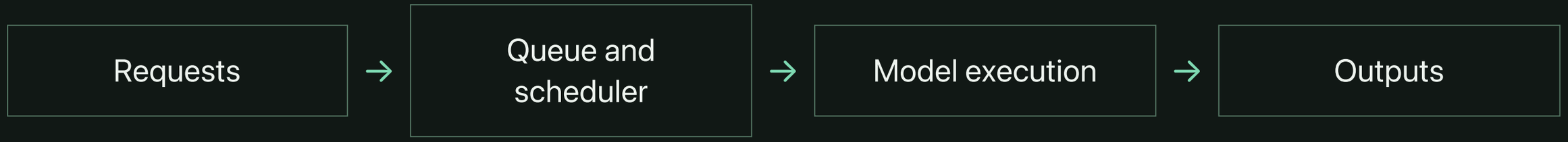
Choose sources, shape the prompt, coordinate calls, and check the result.

Inference engineering levers

Manage model memory, reuse computation, tune batching, and improve execution.

Both affect the 30-second result. Queue time also depends on serving capacity.

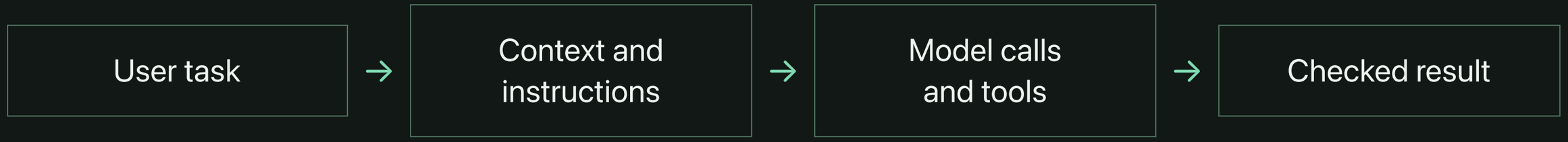
Inside an inference system



Part	How it connects
Model	Its learned weights define the computation needed for a response.
Runtime	Runs that computation using optimized operations on the hardware.
Hardware and memory	Hold model weights and request state, and perform the computation.
Cache	Stores reusable computation, which can reduce repeated work.
Serving system	Routes requests across model replicas and manages available capacity.

Kiely, *Inference Engineering* (2026), chapters 2–5 and 7.
A simplified map of the parts and their relationships.

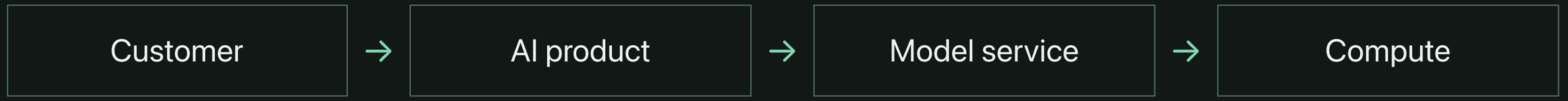
Inside an applied AI system



Part	How it connects
Task and success criteria	Define the useful outcome and how to judge it.
Context	Supplies instructions, retrieved sources, and working history.
Tools and state	Read or change external systems and retain progress.
Orchestration	Coordinates model calls, tool results, retries, and stopping.
Evaluation and controls	Check result quality and constrain which actions are allowed.

Examples: Cursor, OpenCode, and Factory coordinate coding work; CodeRabbit reviews code; Harvey supports legal work.
Product examples from the earlier company slide; this system map is our explanation.

Where the businesses fit



What is supplied	Examples from our company table	What the buyer needs
A useful application	Cursor · OpenCode · Factory Droid CodeRabbit · Harvey	The work completed well.
Managed model access	OpenCode Zen · Factory managed inference	Access to models through a service.
Model serving	Baseten · Together AI · Fireworks · Groq	Reliable model responses at the required scale.
Compute and infrastructure	Modal · Groq	Capacity to run the workload.

One company can supply several layers. Cursor combines an application with its own model and serving work.

Selected product roles, not exclusive company categories. Sources: official product pages cited on the company slide.

Hardware

How compute, memory, and bandwidth shape what a system can do.

What is a GPU?

A graphics processing unit is a processor built to do many similar calculations at the same time. That makes it useful for the matrix operations in AI models.

One operation, many pieces of data

2	4	6
× 3	× 3	× 3
6	12	18

Compute is the calculation capacity: how much arithmetic the processor can perform per second.

A central processing unit (CPU) handles general program logic; a GPU accelerates large amounts of parallel arithmetic.

Kiely, *Inference Engineering* (2026), §§3.1–3.1.1, pp. 74–75.
Illustrative arithmetic, not a chip diagram or benchmark.

Memory: does the work fit?

The model's weights are the numbers learned during training. Memory must hold those weights and the working state of active requests.

Illustration: a model with 7 billion weights, stored at 16 bits each

7 billion weights × 2 bytes per weight = About 14 GB for weights alone

Model weights + Active request state + Runtime workspace → Total memory needed

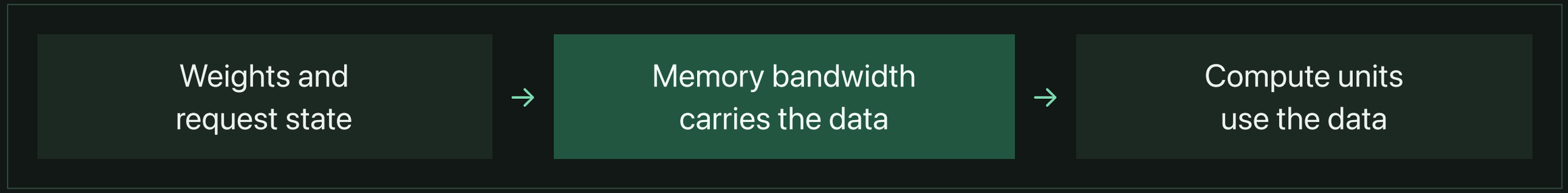
Longer context and more simultaneous requests need more working memory.

More memory can hold a larger batch, but the batch must still meet each user's time limit.

Adapted from the book's sizing rule: Kiely, *Inference Engineering* (2026), §3.1.2, pp. 76–77; §5.4, p. 142.
The 14 GB estimate excludes request state, workspace, and storage overhead.

Bandwidth: can data arrive fast enough?

Memory capacity tells us how much fits. Memory bandwidth tells us how many bytes can move between memory and computation each second.



The book's example	H100	H200
Memory capacity	80 GB	141 GB
Memory bandwidth	3.35 TB per second	4.8 TB per second
8-bit dense compute	1,979 trillion operations per second	1,979 trillion operations per second

More bandwidth can speed up each request; batching can reuse weight reads to serve more requests.

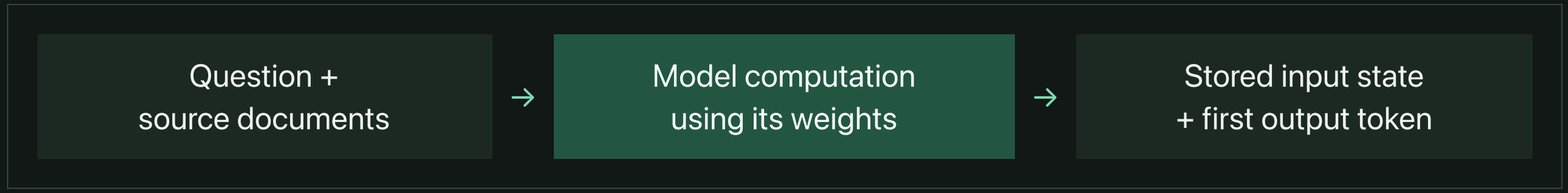
Kiely, *Inference Engineering* (2026), §3.1.2, pp. 76–77; §3.2.1, p. 79.
Book configuration specifications, not measured model speeds. GB = gigabytes; TB = terabytes.

Prefill and decode

How the model processes an input and produces an answer.

Prefill: process the input

Before answering, the model processes the question and source documents. It can work on many input tokens together.



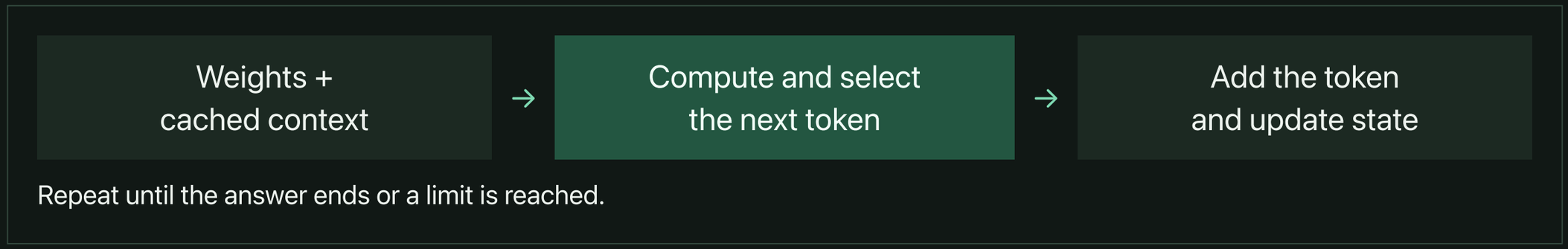
Resource	What prefill needs
Compute	Many calculations across the input tokens.
Memory	Room for the weights, working data, and key-value cache.
Bandwidth	Movement of weights and working data through memory.

The key-value cache stores intermediate state that generation can reuse.

Kiely, *Inference Engineering* (2026), §2.2, pp. 47–49; §2.4.2, pp. 63–66.
Prefill is often compute-limited; the actual workload and implementation determine the bottleneck.

Decode: produce the answer

After the first output token, ordinary generation repeats: use the existing context, choose the next token, and extend the stored state.

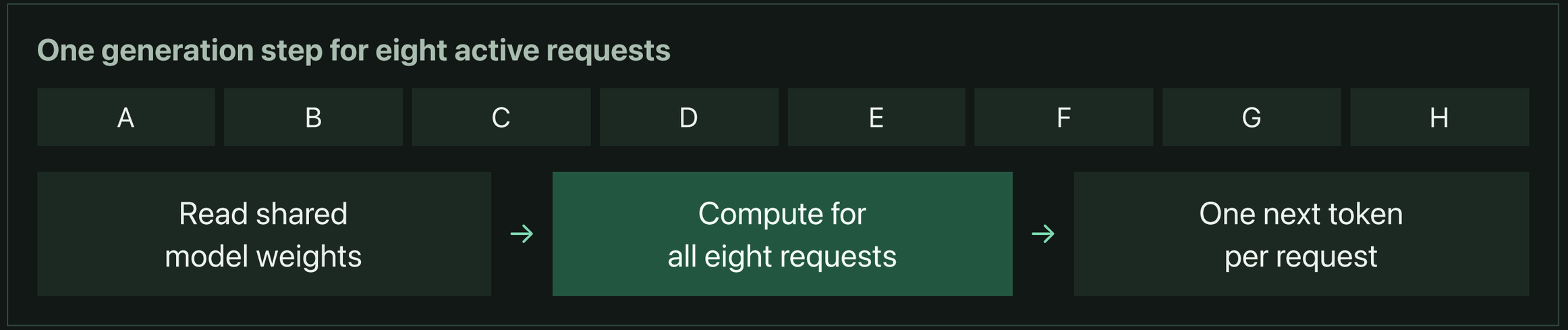


Resource	What decode needs
Compute	Run the model for each new token.
Memory	Keep the weights and growing state of active requests.
Bandwidth	Keep supplying weights and cached state to the computation.

At small batch sizes, moving data can dominate. Larger batches reuse weights, but each request keeps its own state.

One GPU, many requests

Batching lets one graphics processing unit (GPU) work on several requests together.



Hardware resource	What batching changes	Effect on the service
Compute and bandwidth	Reuse weights across more calculations.	More total tokens per second.
Memory capacity	Keep separate state for every request.	Limits how large the batch can be.
Time per step	Do more work before the step finishes.	Each user can receive tokens more slowly.

Simplified decode example, not a benchmark. Kiely, *Inference Engineering* (2026), §2.4, pp. 61–66; §7.2.1, pp. 186–188.

Find what the processor is waiting for

The useful lever depends on whether the workload needs more calculation, faster data movement, or more space.

Limit	What happens	Relevant levers
Compute	The arithmetic units are busy.	Efficient kernels, lower precision, or more suitable compute.
Memory bandwidth	Arithmetic waits for data to arrive.	Reduce data movement, reuse work, or improve bandwidth.
Memory capacity	Weights and active requests do not fit.	Reduce memory use or spread the model across devices.



Prefill often needs substantial compute. Decode at low batch sizes often waits on memory bandwidth.

Both phases sit inside our 30 seconds

Our example budget	Time	Which levers matter?
Retrieve, queue, validate, and send	6 seconds	Retrieval, orchestration, ready capacity, and application code.
Prefill: process the input	4 seconds	Relevant context, prefix reuse, and efficient computation.
Decode: generate the report	14 seconds	Required output length, memory bandwidth, batching, and speculation.
Headroom for variation	6 seconds	Check slow requests and the full workload under load.

Producing about 1,000 tokens in 14 seconds requires about **71 tokens per second** for that response.
A quick first token is useful, but our promise concerns the **complete report**.

Illustrative budget, not measured performance. 6 + 4 + 14 = 24 seconds of work, plus 6 seconds of headroom.
Kiely, *Inference Engineering* (2026), §1.4, pp. 35–37; §2.4, pp. 61–66.

Inference engineering levers

Change how the model runs while preserving useful results.

Optimize model serving

This is called inference optimization, and it is in high demand.
As AI usage grows, products need fast, reliable responses at a sustainable cost.

Where time is spent	What we can change	What we must check
Waiting for capacity	Keep workers ready; tune routing and batching.	Queue delay under normal and peak demand.
Processing the input	Reuse cached prefixes; improve input-processing efficiency.	Input-processing time on cache hits and misses.
Generating the report	Test efficient runtimes, precision, and speculative decoding.	Generation time and quality at the agreed load.

1,000 output tokens in 14 seconds requires about **71 tokens per second per response**.
We must test that rate while serving the agreed workload.

Kiely, *Inference Engineering* (2026), preface, pp. 12–13; §1.4, pp. 35–37; §4.5, pp. 112–114.
These are changes to test, not guaranteed savings.

Latency and throughput

Latency

How long one request takes.
For streaming, also measure when tokens arrive.

Throughput

How much work the whole system completes per second.

Same processor capacity	Tokens per second, per user	Tokens per second, total
Serve 1 request at a time	100	100
Serve 8 requests together	40	320

Why each user gets tokens more slowly

A larger batch does more work in each generation step, so the step can take longer.

Why the system produces more tokens

One read of the model weights serves several requests. Each step produces a token for each request.

Illustrative numbers, not a benchmark. Kiely, *Inference Engineering* (2026), §1.2.2, pp. 29–30; §1.4, pp. 35–37.

One user: response time and quality

Does this person get a useful result soon enough?

Workload	What the user needs	What we measure
Code completion	A useful suggestion before they type past it.	Time to a complete suggestion; acceptance rate.
Chat	A correct answer that starts promptly and streams smoothly.	Time to first token; gaps between tokens; answer quality.
Coding agent	A correct code change that passes the checks.	Total task time; task success.
Document research	A supported report within 30 seconds.	Complete-report time; source support.

Follow one request from start to useful result.

Kiely, *Inference Engineering* (2026), §1.4, pp. 35–37. Examples adapted to our workloads.

Many users: throughput, reliability, and cost

Can we deliver the same experience as demand grows?

Same workload	What the service must do	What we measure across requests
Code completion	Keep suggestions fast during busy periods.	Useful suggestions per second; response-time percentiles; cost.
Chat	Keep many conversations responsive at once.	Total output tokens per second; delays; errors; cost per conversation.
Coding agent	Complete many tasks without long queues.	Successful tasks per hour; queue time; failure rate; cost per task.
Document research	Keep the 30-second promise as requests arrive.	Reports per minute; share within 30 seconds; failure rate; cost per report.

More work per second helps only if users still get correct results on time.

Kiely, *Inference Engineering* (2026), §1.2.2, pp. 29–30; §1.4, pp. 35–37; §4.5, pp. 112–114.

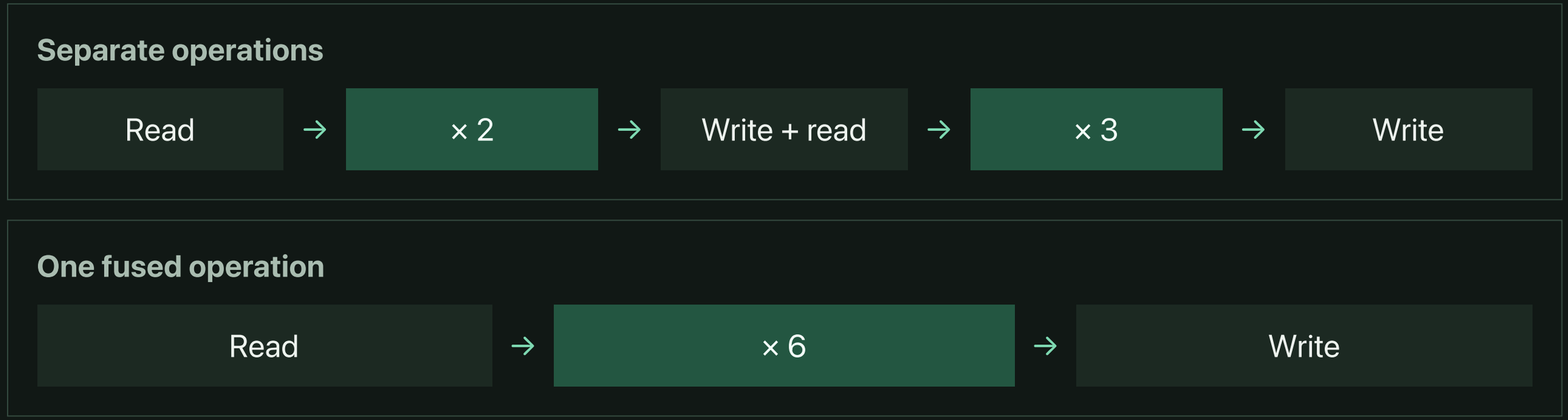
The levers of inference engineering

Lever	What it changes	What can limit the benefit
Runtime and kernels	How efficiently the hardware executes operations.	Model and hardware support.
Numerical precision	Memory use and computation through quantization.	Quality loss and hardware support.
Caching	How much existing computation can be reused.	Cache misses and memory pressure.
Batching and scheduling	Which requests run together and when.	Queue delay and memory limits.
Speculative decoding	Drafts tokens, then verifies them with the target model.	Rejected drafts and verification overhead.
Hardware and model placement	Compute, memory, and how work is split across devices.	Cost and communication between devices.
Prefill/decode disaggregation	Run the two phases on separate workers.	State transfer, extra capacity, and queues.
Replicas and routing	Serving capacity and where each request runs.	Startup time, idle cost, and load imbalance.

Levers of inference engineering

Runtime and kernels: move less data

A kernel performs an operation on the processor. Kernel fusion combines operations so intermediate results do not need a round trip to memory.



The book's example has the same result: [1, 2, 3] → [6, 12, 18].

What changes: memory traffic and execution overhead. Check that the runtime supports the model and hardware.

Kiely, *Inference Engineering* (2026), §4.1.3, pp. 100–101. Diagram redrawn from the book's example.

Levers of inference engineering

Quantization: use fewer bits

Represent model values with fewer bits, so they take less memory and can require less data movement.

The book illustrates the risk with repeated multiplication

3.14159 → Cube the number → **31.006198**

3.14 → Cube the number → **30.959144**

Small precision errors can grow through repeated computation.

What changes: model size, memory traffic, and supported arithmetic.
Compare answer quality and speed with the original model before adopting it.

Kiely, *Inference Engineering* (2026), §5.1, pp. 120–121; §5.1.3, p. 128.
Decimal rounding illustrates error; it is not a diagram of an actual quantized number format.

Levers of inference engineering

Caching: reuse a shared beginning

Prefix caching reuses work for the identical beginning of a prompt.

First request: process the full prompt and retain its state

Weather in	Toronto?

Next request: reuse the shared prefix, then process what changed

Weather in	Waterloo?

For our research service, shared instructions or document context can avoid repeated input processing.

What changes: work done before generation starts.
The matching prefix must be cached; changed input and cache misses still need processing.

Kiely, *Inference Engineering* (2026), §5.3.1, pp. 136–138.
Adapted from the book's “Weather in” example; boxes show text segments, not exact tokens.

Levers of inference engineering

Batching: fill slots as work finishes

The processor can serve several requests together. Continuous batching lets new requests join when a slot becomes free.

Fixed batch

A	A	—	—
B	B	B	B
C	C	C	—

New requests wait for the batch to finish.

Continuous batch

A	A	D	D
B	B	B	B
C	C	C	E

D and E enter the freed slots.

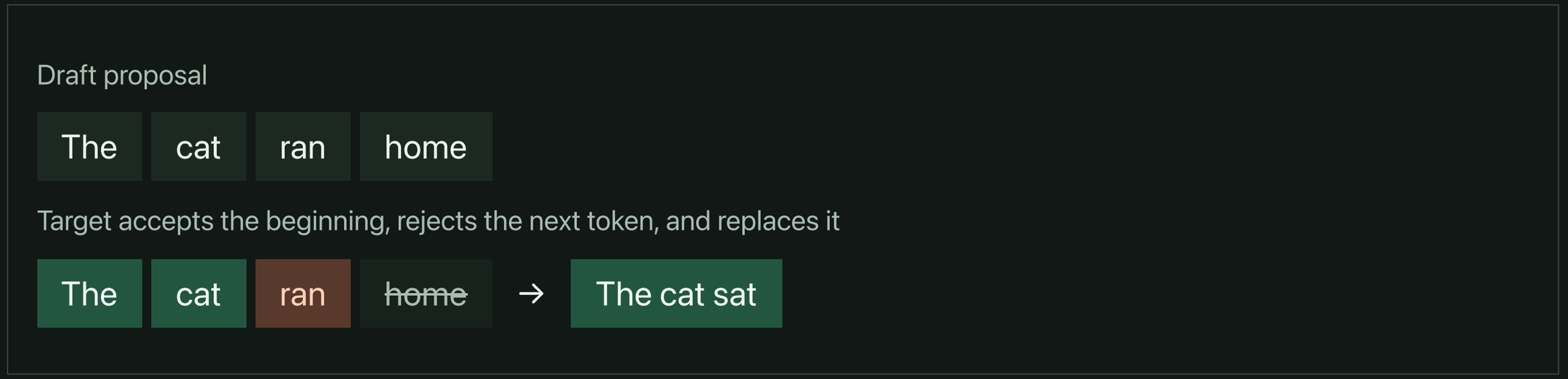
Each row is a request slot; each column is an execution step.

What changes: how requests share the processor.
More work per second can come with slower responses for each user.

Levers of inference engineering

Speculation: draft, then verify

A cheaper draft process proposes several tokens. The target model verifies the draft in one pass.



The book compares generating a token to solving a puzzle, and verification to checking a solution.

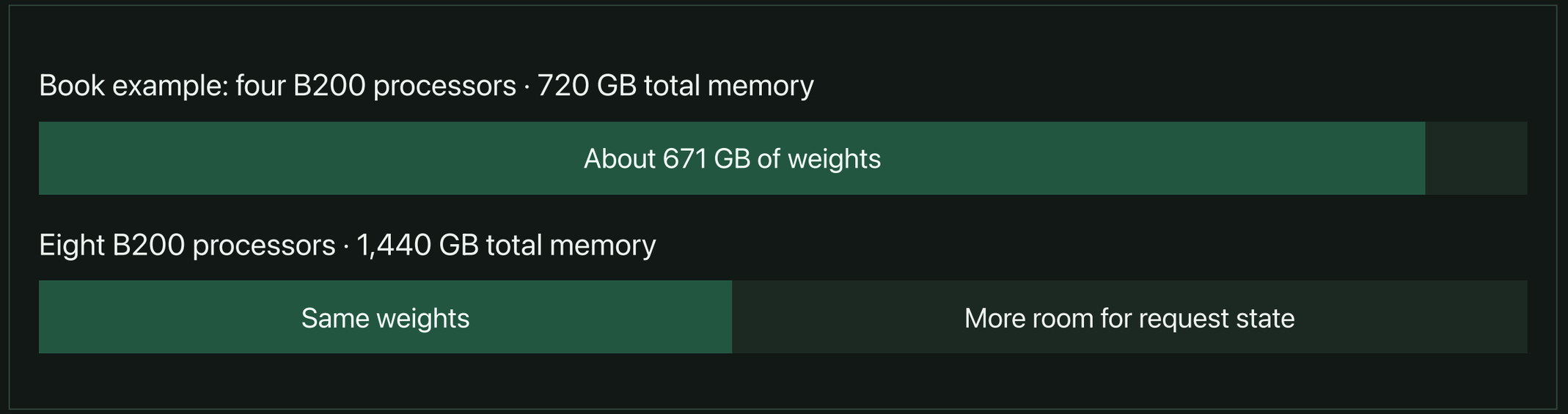
What changes: how many tokens each target-model pass can commit.
The draft must be cheap and accepted often enough to repay its overhead.

Kiely, *Inference Engineering* (2026), §5.2, pp. 129–132.
Toy text illustrates the book's draft-and-verify mechanism.

Levers of inference engineering

Model placement: make room to work

A model needs memory for both its weights and the state of active requests. Splitting it across processors can provide more room.



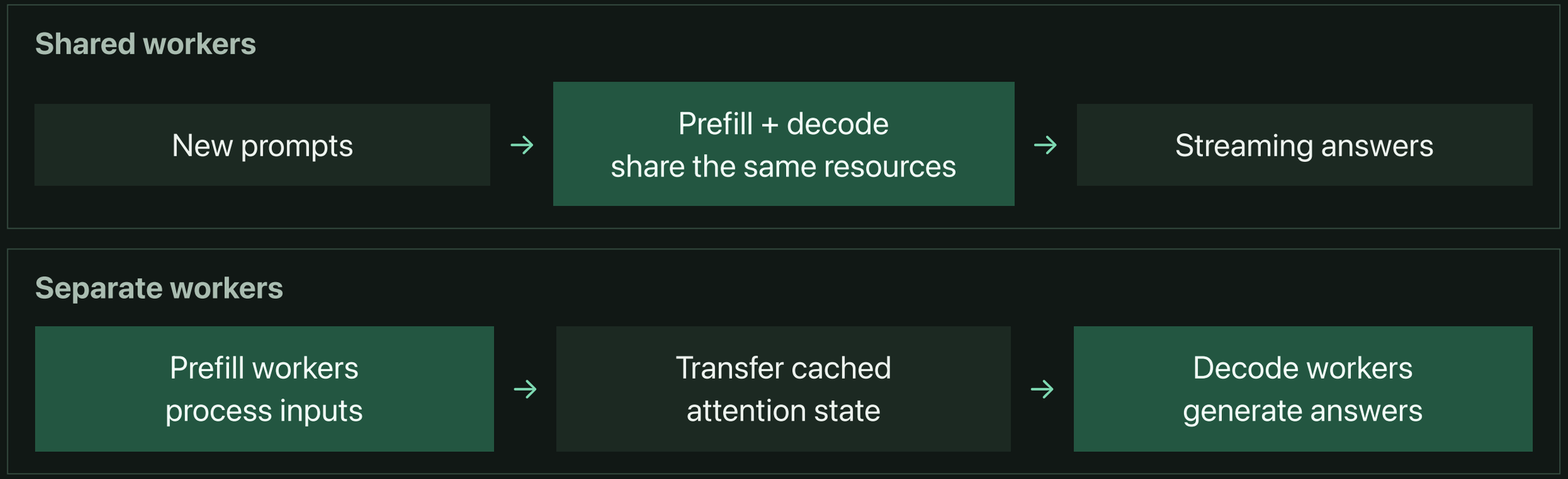
The book's DeepSeek-V3.1 example fits the weights on four processors, but uses eight to serve a practical workload.

What changes: memory capacity and how computation is shared.
Processors must exchange results; communication can erase the speed benefit.

Levers of inference engineering

Disaggregation: separate the two phases

Under heavy load, a new long prompt can compete with requests already generating answers. Separate workers can reduce that interference.



Tune and scale each pool separately to protect response times while serving more requests.

The cost: extra workers and state transfer. It can lose to shared workers at low load or with short, cached inputs.

Levers of inference engineering

Replicas and routing: send work wisely

A replica is another running copy of the model. Routing decides which copy gets each request.

Replica A · four requests

4 × 100
input tokens

400 input tokens to process.

Replica B · four requests

3 × 100 + 10,000
input tokens

10,300 input tokens to process.

Equal request counts do not mean equal work. Route with workload size, queue state, and cached context in mind.

What changes: where requests run and how much capacity is available.
New replicas take time to start, and idle replicas still cost money.

Kiely, *Inference Engineering* (2026), §7.2, pp. 183–191; §5.3.3, pp. 140–141.
Adapted from the book's four-request and 100-versus-10,000-token routing example.

Applied AI levers

Change the work the application asks the model to do.

Levers of applied AI

Evaluation: check the useful outcome

Define what success means, then test changes against it before judging speed or cost.



Task	What success requires
Contract research	The correct deadline, supported by the applicable clause.
Book a meeting	The intended event exists with the correct people and time.
Review code	Real issues found, with acceptable false alarms.

Inference connection: a quality baseline lets us test smaller models, quantization, and other optimizations without silently breaking the task.

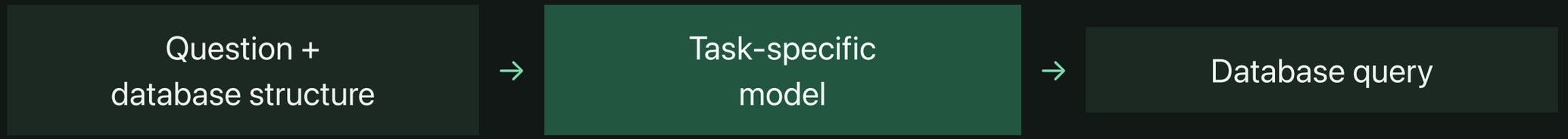
Kiely, *Inference Engineering* (2026), §1.3.1, pp. 31–32; §5.1.3, p. 128.
Task criteria are workshop examples; a successful response alone is not proof of a successful action.

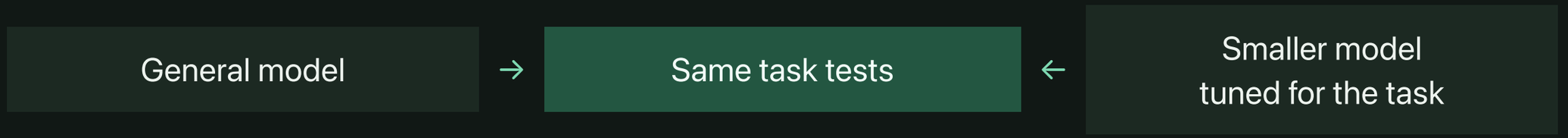
Levers of applied AI

Model choice: match the task

Choose a model that meets the quality requirement, then compare how much it costs to run.

The book's example: turn a question into a database query





Compare both candidates on representative questions and difficult cases.

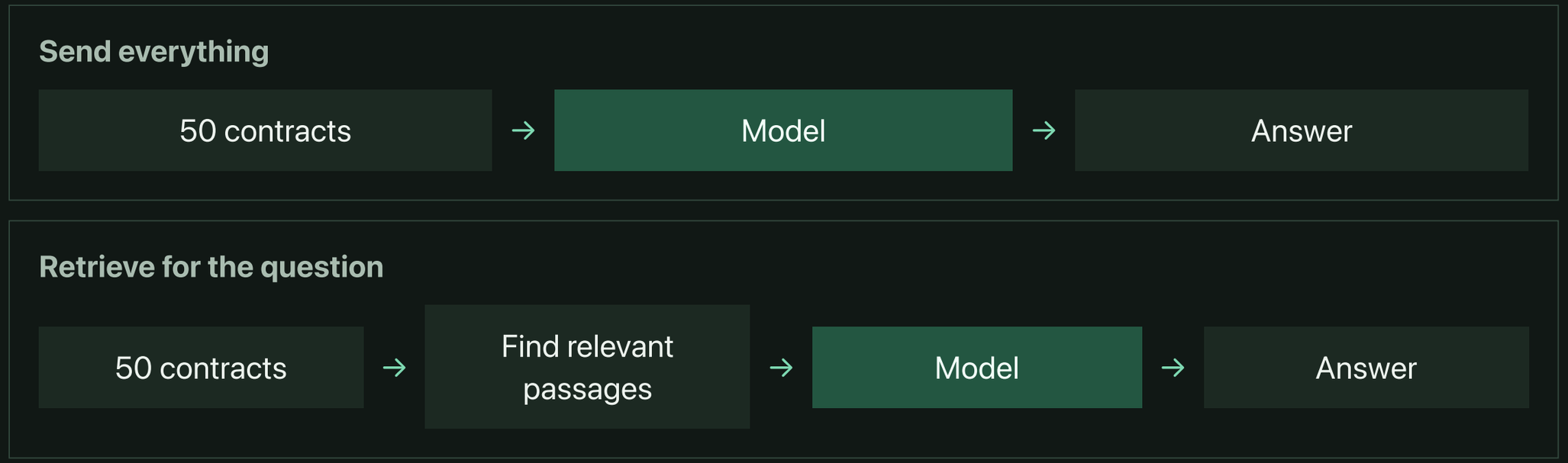
The book describes a small, fine-tuned model matching a much larger model on a narrow text-to-query task.

Inference connection: model size and architecture change memory needs, compute, and available optimizations.

Levers of applied AI

Context: send the useful evidence

Choose which information reaches the model. Retrieval finds relevant material before the model writes its answer.



Example: a question about cancellation needs the applicable cancellation terms and their source references.

Inference connection: fewer input tokens can reduce input processing and memory use. Missing evidence can still make the answer wrong.

Levers of applied AI

Instructions: define the result needed

State the task, the evidence to use, and the output the application needs.

Vague request

“Tell me about this contract.”

The model must guess the scope and level of detail.

Task-specific request

“Find the cancellation deadline. Cite the clause. If it is absent, say ‘not found.’”

The application asks for a result it can use and check.

What changes: the requested behavior, answer shape, and stopping point.

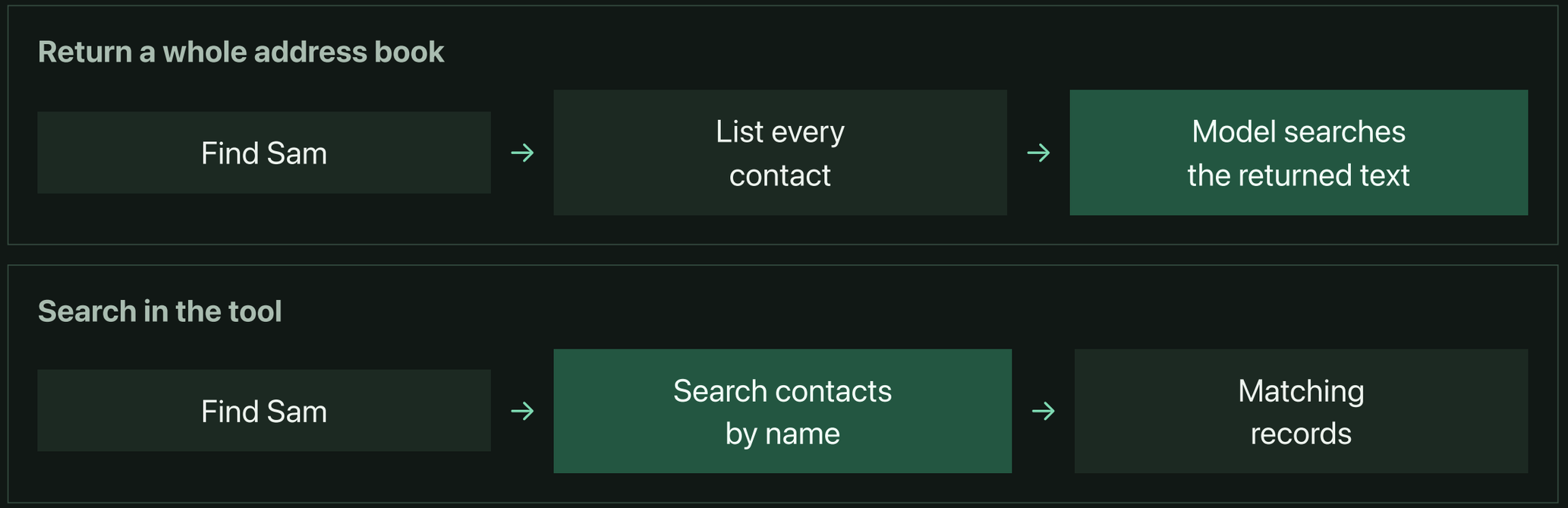
Inference connection: output length changes generation work. Stable instructions can also form a reusable prompt prefix.

Illustrative prompts. Anthropic, Effective context engineering for AI agents.
Prefix reuse: Kiely, *Inference Engineering* (2026), §5.3.1, pp. 136–138.

Levers of applied AI

Tools: do precise work outside the model

Give the model a clear operation that retrieves or changes exactly what the task needs.



What changes: where filtering happens and how much information returns to the model.

Inference connection: focused tool results can reduce the next prompt. Clear tool interfaces can also reduce failed calls and retries.

Levers of applied AI

State: preserve progress between calls

Store completed work and its evidence so the next call can continue from the right place.

Repeat the full history

Old prompts, repeated searches, raw documents, and every intermediate response.

The next call must find the current task state again.

Keep a task record and source links

Completed: contracts A and B

Pending: contract C

Evidence: clause references

Next: compare the deadlines

What changes: the information carried forward and the work that must be repeated.

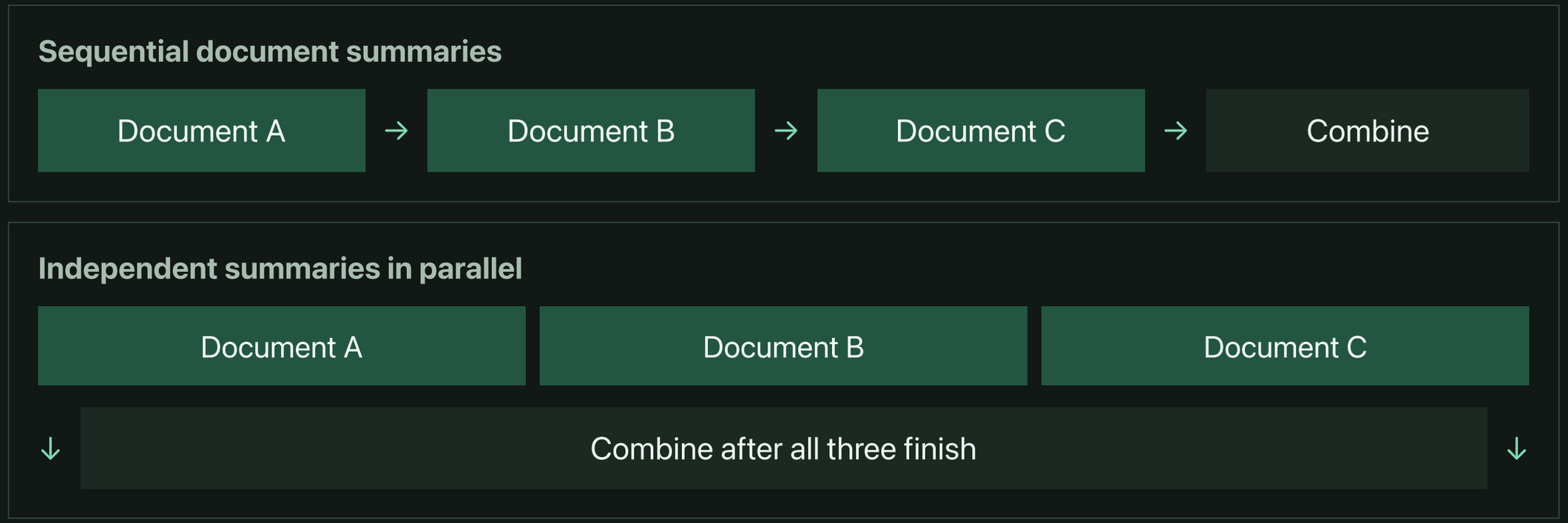
Inference connection: a focused working context can reduce input processing. Preserve critical facts and measure cache reuse after changes.

Illustrative task record. State and compaction: Anthropic, Effective context engineering for AI agents.
Cache behavior: Kiely, *Inference Engineering* (2026), §5.3.1, pp. 136–138.

Levers of applied AI

Orchestration: change the order of work

Run independent steps together. Keep a step sequential when it needs an earlier result.



What changes: the longest chain of dependent work, not necessarily the number of model calls.

Inference connection: parallel calls create more simultaneous demand. Queueing can consume the time you hoped to save.

Putting it together

Use both sets of levers to meet the same time, quality, and cost requirements.

Back to the 30-second promise

Suppose a request takes 32 seconds. Where should we intervene?

Stage	Example time	What we investigate
Retrieve sources	3 seconds	Did we find the evidence the report needs?
Wait for capacity	7 seconds	Were workers busy or still starting?
Process the input	6 seconds	How much context was sent? Was any prefix reused?
Generate the report	15 seconds	How much output was needed? How fast did it arrive?
Validate and send	1 second	Was the report complete and correctly formed?

A 32-second request misses the latency target. One request does not determine the monthly result.

Illustrative timings, not benchmark data. Same service limits: 16,000 input tokens, 1,000 output tokens, 20 requests in flight.
Kiely, *Inference Engineering* (2026), §1.4, pp. 35–37; §4.5, pp. 112–114.

Pair the two sets of levers

Applied AI shapes the workload; inference engineering changes how it runs.

Problem	Applied AI change	Inference engineering change
Too much input work	Retrieve relevant clauses; put stable instructions before changing content.	Reuse matching prefixes and profile input processing.
Long queue waits	Remove redundant calls; limit simultaneous internal calls.	Keep enough capacity ready; tune routing and batching.
Slow report generation	Define the required report; evaluate smaller models on the same task.	Test efficient runtimes, precision, and speculative decoding.
Fast but unreliable answers	Check source support, missing facts, and actual task success.	Reject optimizations that break the quality baseline.

Change one cause at a time, then test quality, time, and cost across the full workflow.

Worked example using Kiely, *Inference Engineering* (2026), §§1.3–1.4, chapters 4–5, and §7.2.

Batching within the 30-second promise

Use when: several report jobs overlap and the processor has room to serve them together.

Measure	What can improve	What can go wrong
Latency	Higher capacity can shorten the queue.	Waiting to fill a batch or slower generation can delay each report.
Throughput	Shared weight reads can produce more tokens per second overall.	A batch that is too large can exhaust memory or miss the time limit.
Cost	More completed reports per processor-hour can lower cost per report.	Extra retries or late reports can erase the saving.

Start with continuous batching, then tune the active batch and queue limits under the agreed load.

Our check: retain the 24-second operating target and 6-second margin; measure complete reports with up to 20 jobs in flight.

Candidate strategy, not measured savings. Kiely, *Inference Engineering* (2026), §1.4, pp. 35–37; §7.2.1, pp. 186–188.

When would we separate prefill and decode?

Use when: long new inputs repeatedly slow ongoing generation, and traffic is high enough to keep both worker pools useful.

Measure	What can improve	What can go wrong
Latency	Protect the report's generation from competing input processing.	State transfer and two queues can add more time than they save.
Throughput	Batch, tune, and scale each phase for its share of the work.	Too few workers in either pool can limit the whole service.
Cost	Better use of busy workers can lower cost per report at scale.	Extra workers, idle capacity, and data transfer can cost more.

Our 4-second prefill and 14-second decode budgets must also absorb the new transfer and queues.

Decision: compare shared workers against separate pools under the same traffic. Twenty jobs in flight alone is not a reason to split.

Kiely, *Inference Engineering* (2026), §5.5, pp. 148–151. With a hosted model, compare provider offerings; these are serving-side controls.

Aim for 24 seconds, then test

Stage	Slow example	Candidate budget
Retrieve sources	3 seconds	3 seconds
Wait for capacity	7 seconds	2 seconds
Process the input	6 seconds	4 seconds
Generate the report	15 seconds	14 seconds
Validate and send	1 second	1 second
Total	**32 seconds**	**24 seconds + 6 seconds of headroom**

Test representative inputs, cache hits and misses, and demand up to the agreed limit.

Keep answer quality intact, then monitor the monthly 95% and 99.9% commitments.

Illustrative budget, not a claimed speedup. A passing load test supports readiness; monthly measurements determine compliance.

Kiely, *Inference Engineering* (2026), §4.5, pp. 112–114; §7.4, pp. 199–203.

Reduce cost with both sets of levers

Same GPT-6.1 Sol model, 100,000 reports per month, and 1,000 output tokens per report.

US$4,200
Original input

→

US$2,600
Focused input

→

US$1,604
Reused prefix

Step	What changes	Lever
Original request	16,000 input tokens for every report.	Our pricing baseline.
Keep relevant evidence	8,000 input tokens, with the same required answer.	Applied AI: retrieval and context design.
Reuse a shared beginning	6,000 stable tokens + 2,000 changing tokens; 90% of requests reuse the prefix.	Applied AI arranges the prompt; inference serving reuses its computation.

Potential model-usage saving: **US$2,596 per month**. The report still has to be correct and on time.

Illustrative workload and cache reuse, not measured savings. Includes 30-minute cache writes.
Rates: OpenAI Standard pricing · Prompt caching, checked November 10, 2026.

Keep the 30-second commitment

Cost-saving change	What could go wrong	What we test
Retrieve less context	A needed clause or exception is omitted.	The same report-quality checks, plus retrieval time.
Reuse a cached prefix	The cache is cold or the shared content changes.	Complete-report latency on both cache hits and misses.
Coordinate simultaneous calls	A burst creates a long queue.	Up to 20 report jobs at once, including the largest allowed inputs.

Keep the same promise: 95% complete within 30 seconds; 99.9% succeed within 60 seconds.
Accept savings only when quality and the complete service still meet their requirements.

With a hosted model, we control the application and service choice; the provider controls the underlying model execution.
Kiely, *Inference Engineering* (2026), §1.3.1, pp. 31–32; §4.5, pp. 112–114.

Workshop

Test a workload, change one lever, and measure the result.

The Bitter Lesson

"We want AI agents that can discover like we can, not which contain what we have discovered."

Richard Sutton, The Bitter Lesson, March 13, 2019.

General search and learning methods can improve with more computation.

If the agent can write the code, what is your role?

Define the goal and success checks → Give the agent tools and context → Let it try and test a change → Use feedback to try again ↻

Example: give the agent a reproducible login bug, expected behavior, and a test runner. Let it find and check the fix.

You define the problem, design the system, and verify that it works for the user.

Same model. Why pay for speed?

GPT-6 Astra Ultrafast: up to **8× faster token generation** in Codex.
8× included allowance, or **6× paid credits**, compared with Standard.

If we serve fewer requests together, per-user latency can **decrease** .

Throughput per processor can **decrease** , so cost per token can **increase** .

Smaller batches can share memory reads across fewer requests.
This is one possible tradeoff, not a confirmed account of OpenAI's implementation.

Published speed and usage: OpenAI, Speed · Pricing, checked November 10, 2026.

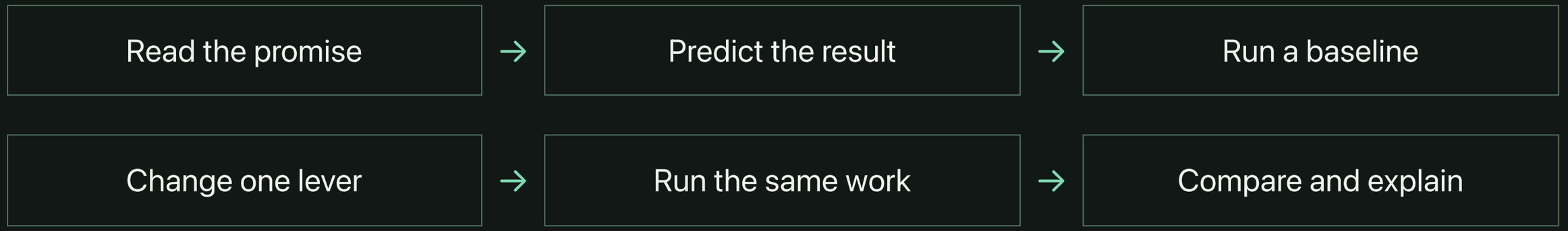
Three experiments, real measurements

Run Qwen2.5 1.5B on your own hardware. Use your coding agent to inspect and change the system.

Experiment	Customer need	What we investigate
What counts as done?	A useful answer, suggestion, or completed action.	Speed and correctness are separate checks.
The service gets busy	The same experience as more people arrive.	Request concurrency, serving capacity, and queues.
Two workloads share hardware	Finish the batch while keeping chat responsive.	Throughput, interference, and cost per correct result.

Open the dashboard your agent started: 127.0.0.1:8878/lab/index.html.
Standard runs use 100 requests; the calendar experiment uses 40 tasks.

Make one change, then measure



Check the output as well as the timing. A faster wrong result still fails the task.

Keep the model and workload fixed when testing a serving change. Save the baseline so you can explain what improved and what got worse.

What counts as done?

Workload	Time target	Quality target
Support chat	95% start within 1 second.	At least 90% use the correct source fact.
Code completion	95% finish within 250 milliseconds.	At least 90% contain the correct code.
Calendar agent	90% finish within 3.5 seconds.	At least 90% save the correct calendar event.

Run a baseline. Change the agent prompt or enable the final-state check, then compare the actual calendar outcomes.

These are targets across the run. A model saying “done” is not proof that the event exists.

The service gets busy

At four users, 95% of answers should start within 1 second, and at least 90% should contain the correct source fact.

One user → Two users → Four users

Change one thing	Compare
Increase users from one to four.	First-token time and complete-answer time.
Keep four users; increase serving slots from one to four.	Response times, total throughput, and correctness.

Client concurrency sends more work. Server slots determine how much work can run together. They are different controls.

Two workloads share the hardware

Procurement team

Extract the reference, agency, and deadline from 100 notices.

Finish within 30 seconds, with at least 90% fully correct.

Chat user

Keep 95% of answers starting within 1 second.

The promise still applies while the batch runs.

Change worker count, shared versus separate work, or JSON output. Change one control per run and compare both workloads.

Cost target: at most US$0.12 per 1,000 correct notices, using an assumed US$1.20 per machine-hour. Check cost and both service promises.

Write the promise for your project

Define	Make it specific
The useful outcome	Who needs it, and what counts as correct?
The workload	Inputs, outputs, calls per task, and demand.
The target	What is measured, where, over what period, and under what load?
The agreement	Who receives the promise, and what happens if it is missed?
The next experiment	Which lever will you change, and what would show that it helped?

Explain your baseline, one change, and its tradeoff to the person next to you.